

1. Personal Profile Page

- Create a responsive personal profile webpage using HTML5 and CSS3. Include sections for bio, skills, education, and contact details. Use Bootstrap for layout and styling.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Personal Profile Page</title>
  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background-color: #f8f9fa;
    }
    .profile-img {
      width: 150px;
      height: 150px;
      object-fit: cover;
      border-radius: 50%;
      border: 4px solid #0d6efd;
    }
    .section-title {
      border-bottom: 2px solid #0d6efd;
      margin-bottom: 1rem;
      padding-bottom: 0.5rem;
    }
  </style>
</head>
<body>

  <div class="container py-5">
    <!-- Header -->
    <div class="text-center mb-4">
      
      <h1 class="mt-3">John Doe</h1>
      <p class="text-muted">Web Developer | Frontend Specialist</p>
    </div>

    <!-- Bio Section -->
    <div class="mb-5">
      <h3 class="section-title">Bio</h3>
      <p>Hello! I'm John, a passionate web developer with a knack for crafting responsive, user-
friendly websites. I specialize in frontend development and have a strong foundation in HTML, CSS,
JavaScript, and React.</p>
    </div>
```

```

<!-- Skills Section -->
<div class="mb-5">
  <h3 class="section-title">Skills</h3>
  <div class="mb-3">
    <label>HTML</label>
    <div class="progress">
      <div class="progress-bar bg-success" style="width: 90%;">90%</div>
    </div>
  </div>
  <div class="mb-3">
    <label>CSS</label>
    <div class="progress">
      <div class="progress-bar bg-info" style="width: 85%;">85%</div>
    </div>
  </div>
  <div class="mb-3">
    <label>JavaScript</label>
    <div class="progress">
      <div class="progress-bar bg-warning" style="width: 80%;">80%</div>
    </div>
  </div>
  <div class="mb-3">
    <label>React</label>
    <div class="progress">
      <div class="progress-bar bg-primary" style="width: 75%;">75%</div>
    </div>
  </div>
</div>

<!-- Education Section -->
<div class="mb-5">
  <h3 class="section-title">Education</h3>
  <ul class="list-group">
    <li class="list-group-item">
      <strong>B.Sc. in Computer Science</strong> – XYZ University (2019–2023)
    </li>
    <li class="list-group-item">
      <strong>High School</strong> – ABC High School (2017–2019)
    </li>
  </ul>
</div>

<!-- Contact Section -->
<div>
  <h3 class="section-title">Contact</h3>
  <ul class="list-unstyled">
    <li><strong>Email:</strong> john.doe@example.com</li>
    <li><strong>Phone:</strong> +1 123-456-7890</li>
    <li><strong>LinkedIn:</strong> <a href="#">linkedin.com/in/johndoe</a></li>
    <li><strong>GitHub:</strong> <a href="#">github.com/johndoe</a></li>
  </ul>

```

```

    </div>
</div>

<!-- Bootstrap JS CDN -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

2. Product Landing Page

- Design a landing page for a product (e.g., headphones, book) with a header, product image, description, pricing, and a call-to-action button. Use Bootstrap grids.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Product Landing Page - Headphones</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background-color: #f2f4f6;
    }
    .hero {
      background-color: #ffffff;
      padding: 60px 0;
      border-radius: 12px;
      box-shadow: 0 4px 10px rgba(0,0,0,0.1);
    }
    .product-image {
      max-width: 100%;
      border-radius: 10px;
    }
    .price {
      font-size: 1.5rem;
      color: #198754;
      font-weight: bold;
    }
  </style>
</head>
<body>

  <!-- Header -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand" href="#">SoundMax</a>
    </div>

```

```

</nav>

<!-- Hero Section -->
<div class="container my-5">
  <div class="row hero align-items-center">
    <div class="col-md-6 mb-4 mb-md-0">
      
    </div>
    <div class="col-md-6">
      <h2>Wireless Noise-Canceling Headphones</h2>
      <p class="lead">Experience the next level of audio clarity with our cutting-edge noise-canceling wireless headphones. Designed for comfort, built for performance.</p>
      <p class="price">$149.99</p>
      <button class="btn btn-success btn-lg mt-3">Buy Now</button>
    </div>
  </div>
</div>

<!-- Footer -->
<footer class="bg-dark text-white text-center py-3">
  &copy; 2025 SoundMax. All rights reserved.
</footer>

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

3. Survey Form

- Build a survey form with fields for name, email, age, dropdown for preferences, radio buttons for ratings, and a submit button. Style it with CSS.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Survey Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f0f4f8;
      padding: 40px;
    }

    .survey-container {
      max-width: 500px;
      margin: 0 auto;
    }
  </style>

```

```
background: white;
padding: 30px;
border-radius: 10px;
box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
```

```
h2 {
  text-align: center;
  color: #333;
}
```

```
label {
  display: block;
  margin-top: 15px;
  font-weight: bold;
}
```

```
input[type="text"],
input[type="email"],
input[type="number"],
select {
  width: 100%;
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #ccc;
  border-radius: 6px;
}
```

```
.radio-group {
  margin-top: 10px;
}
```

```
.radio-group input {
  margin-right: 8px;
}
```

```
button {
  display: block;
  width: 100%;
  margin-top: 20px;
  background: #0d6efd;
  color: white;
  padding: 12px;
  border: none;
  border-radius: 6px;
  font-size: 16px;
  cursor: pointer;
}
```

```
button:hover {
  background: #0b5ed7;
}
```

```

    }
  </style>
</head>
<body>

  <div class="survey-container">
    <h2>Feedback Survey</h2>
    <form>
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" required />

      <label for="email">Email:</label>
      <input type="email" id="email" name="email" required />

      <label for="age">Age:</label>
      <input type="number" id="age" name="age" min="10" max="100" required />

      <label for="preference">Preferred Product Type:</label>
      <select id="preference" name="preference" required>
        <option value="">--Select--</option>
        <option value="headphones">Headphones</option>
        <option value="speakers">Speakers</option>
        <option value="books">Books</option>
        <option value="accessories">Accessories</option>
      </select>

      <label>Rate Your Experience:</label>
      <div class="radio-group">
        <input type="radio" name="rating" value="Excellent" required /> Excellent<br />
        <input type="radio" name="rating" value="Good" /> Good<br />
        <input type="radio" name="rating" value="Average" /> Average<br />
        <input type="radio" name="rating" value="Poor" /> Poor
      </div>

      <button type="submit">Submit Survey</button>
    </form>
  </div>

</body>
</html>

```

4. Restaurant Menu Page

- Create a restaurant menu page with food categories, items, prices, and images. Use Bootstrap cards and grids.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Restaurant Menu</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<style>
  body {
    background-color: #f8f9fa;
  }
  .menu-section {
    margin-top: 50px;
  }
  .card img {
    height: 200px;
    object-fit: cover;
  }
  h2 {
    border-bottom: 2px solid #198754;
    padding-bottom: 10px;
    margin-bottom: 30px;
    color: #198754;
  }
</style>
</head>
<body>

<!-- Header -->
<nav class="navbar navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Gourmet Bites</a>
  </div>
</nav>

<div class="container menu-section">
  <!-- Starters -->
  <h2>Starters</h2>
  <div class="row g-4">
    <div class="col-md-4">
      <div class="card">
        
        <div class="card-body">
          <h5 class="card-title">Bruschetta</h5>
          <p class="card-text">Grilled bread topped with tomato and basil.</p>
          <p class="text-success fw-bold">$6.99</p>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card">
        

```

```
<div class="card-body">
  <h5 class="card-title">Garlic Mushrooms</h5>
  <p class="card-text">Sautéed mushrooms in garlic butter.</p>
  <p class="text-success fw-bold">$5.99</p>
</div>
</div>
</div>
</div>
```

```
<!-- Main Course -->
<h2 class="mt-5">Main Course</h2>
<div class="row g-4">
  <div class="col-md-4">
    <div class="card">
      
      <div class="card-body">
        <h5 class="card-title">Grilled Salmon</h5>
        <p class="card-text">Fresh salmon fillet with herbs and lemon.</p>
        <p class="text-success fw-bold">$15.99</p>
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="card">
      
      <div class="card-body">
        <h5 class="card-title">Veggie Pasta</h5>
        <p class="card-text">Penne pasta with mixed seasonal vegetables.</p>
        <p class="text-success fw-bold">$12.99</p>
      </div>
    </div>
  </div>
</div>
</div>
```

```
<!-- Desserts -->
<h2 class="mt-5">Desserts</h2>
<div class="row g-4 mb-5">
  <div class="col-md-4">
    <div class="card">
      
      <div class="card-body">
        <h5 class="card-title">Chocolate Cake</h5>
        <p class="card-text">Rich and moist chocolate delight.</p>
        <p class="text-success fw-bold">$4.99</p>
      </div>
    </div>
  </div>
</div>
<div class="col-md-4">
```

```

<div class="card">
  
  <div class="card-body">
    <h5 class="card-title">Classic Cheesecake</h5>
    <p class="card-text">Creamy New York-style cheesecake.</p>
    <p class="text-success fw-bold">$5.49</p>
  </div>
</div>
</div>
</div>
</div>

```

```

<!-- Footer -->
<footer class="bg-dark text-white text-center py-3">
  &copy; 2025 Gourmet Bites. All rights reserved.
</footer>

```

```

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

5. Responsive Blog Layout

- Design a blog homepage with a header, navigation bar, featured post section, and article grid. Ensure it is mobile-friendly.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>My Blog</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    .featured {
      background: linear-gradient(to right, #1e3a8a, #10b981);
      color: white;
      padding: 60px 30px;
      border-radius: 10px;
    }
    .featured h1 {
      font-size: 2.5rem;
    }
    .post-card img {
      height: 180px;
      object-fit: cover;
    }
  </style>

```

```

    footer {
      background: #343a40;
      color: white;
      padding: 15px 0;
      text-align: center;
    }
  </style>
</head>
<body>

<!-- Header -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">My Blog</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Articles</a></li>
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>

<!-- Featured Post -->
<section class="featured text-center my-5">
  <div class="container">
    <h1>Welcome to My Blog</h1>
    <p class="lead">Explore articles on technology, lifestyle, and creativity.</p>
  </div>
</section>

<!-- Article Grid -->
<div class="container my-5">
  <div class="row g-4">
    <!-- Blog Post 1 -->
    <div class="col-md-4">
      <div class="card post-card">
        
        <div class="card-body">
          <h3 class="card-title">Exploring AI in 2025</h3>
          <p class="card-text">A look at the latest trends in artificial intelligence and its impact on our
lives.</p>
          <a href="#" class="btn btn-outline-primary btn-sm">Read More</a>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    </div>
  </div>

  <!-- Blog Post 2 -->
  <div class="col-md-4">
    <div class="card post-card">
      
      <div class="card-body">
        <h5 class="card-title">Minimalist Lifestyle</h5>
        <p class="card-text">How embracing minimalism can lead to a more fulfilling life.</p>
        <a href="#" class="btn btn-outline-primary btn-sm">Read More</a>
      </div>
    </div>
  </div>

  <!-- Blog Post 3 -->
  <div class="col-md-4">
    <div class="card post-card">
      
      <div class="card-body">
        <h5 class="card-title">Creative Writing Tips</h5>
        <p class="card-text">Improve your storytelling with these essential writing techniques.</p>
        <a href="#" class="btn btn-outline-primary btn-sm">Read More</a>
      </div>
    </div>
  </div>

  <!-- Footer -->
  <footer>
    <div class="container">
      <p>&copy; 2025 My Blog. All rights reserved.</p>
    </div>
  </footer>

  <!-- Bootstrap JS -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

6. To-Do List App

- Implement a to-do list where users can add, delete, and mark tasks as complete. Use JavaScript for DOM manipulation.

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>To-Do List App</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f3f4f6;
      margin: 0;
      padding: 20px;
    }
    .container {
      max-width: 500px;
      margin: auto;
      background: #fff;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    h2 {
      text-align: center;
      color: #1e3a8a;
    }
    input[type="text"] {
      width: 70%;
      padding: 10px;
      font-size: 16px;
    }
    button {
      padding: 10px;
      font-size: 16px;
      background: #10b981;
      border: none;
      color: white;
      cursor: pointer;
    }
    ul {
      list-style-type: none;
      padding: 0;
      margin-top: 20px;
    }
    li {
      background: #e5e7eb;
      padding: 10px;
      margin-bottom: 10px;
      border-radius: 5px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      transition: all 0.3s;
    }
  </style>
</html>
```

```

}
li.completed {
  background: #d1fae5;
  text-decoration: line-through;
  color: #6b7280;
}
.actions button {
  background: #ef4444;
  border: none;
  color: white;
  padding: 5px 10px;
  margin-left: 5px;
  cursor: pointer;
  border-radius: 3px;
}
.actions .complete {
  background: #3b82f6;
}
</style>
</head>
<body>
<div class="container">
  <h2>To-Do List</h2>
  <div>
    <input type="text" id="taskInput" placeholder="Enter a new task...">
    <button onclick="addTask()">Add</button>
  </div>
  <ul id="taskList"></ul>
</div>

<script>
function addTask() {
  const input = document.getElementById("taskInput");
  const taskText = input.value.trim();
  if (taskText === "") return;

  const li = document.createElement("li");

  const span = document.createElement("span");
  span.textContent = taskText;
  span.style.flexGrow = "1";

  const actions = document.createElement("div");
  actions.className = "actions";

  const completeBtn = document.createElement("button");
  completeBtn.className = "complete";
  completeBtn.textContent = "✔";
  completeBtn.onclick = () => li.classList.toggle("completed");

  const deleteBtn = document.createElement("button");

```

```

deleteBtn.textContent = "🗑️";
deleteBtn.onclick = () => li.remove();

actions.appendChild(completeBtn);
actions.appendChild(deleteBtn);

li.appendChild(span);
li.appendChild(actions);

document.getElementById("taskList").appendChild(li);
input.value = "";
}
</script>
</body>
</html>

```

7. Simple Calculator

- Build a calculator with basic operations (+, -, *, /) using HTML, CSS, and JavaScript.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Simple Calculator</title>
  <style>
    body {
      background-color: #f3f4f6;
      font-family: Arial, sans-serif;
      display: flex;
      height: 100vh;
      justify-content: center;
      align-items: center;
    }
    .calculator {
      background: #ffffff;
      padding: 20px;
      border-radius: 15px;
      box-shadow: 0 0 15px rgba(0,0,0,0.2);
      width: 300px;
    }
    .display {
      width: 100%;
      height: 50px;
      text-align: right;
      font-size: 24px;
      margin-bottom: 15px;
      padding: 10px;
      border: 1px solid #ccc;
      border-radius: 10px;
    }
  </style>
</head>
<body>
  <div class="calculator">
    <div class="display"></div>
    <div>
      <input type="text" />
      <div>
        <button>+</button>
        <button>-</button>
        <button>*</button>
        <button>/</button>
      </div>
    </div>
  </div>
</body>

```

```

}
.buttons {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
}
button {
  padding: 20px;
  font-size: 18px;
  border: none;
  border-radius: 10px;
  background: #10b981;
  color: white;
  cursor: pointer;
  transition: 0.2s;
}
button.operator {
  background-color: #3b82f6;
}
button.equal {
  background-color: #1e3a8a;
}
button.clear {
  background-color: #ef4444;
}
button:hover {
  filter: brightness(1.1);
}
</style>
</head>
<body>

<div class="calculator">
  <input type="text" class="display" id="display" disabled />
  <div class="buttons">
    <button onclick="append('7')">7</button>
    <button onclick="append('8')">8</button>
    <button onclick="append('9')">9</button>
    <button class="operator" onclick="append('/')">÷</button>

    <button onclick="append('4')">4</button>
    <button onclick="append('5')">5</button>
    <button onclick="append('6')">6</button>
    <button class="operator" onclick="append('*')">x</button>

    <button onclick="append('1')">1</button>
    <button onclick="append('2')">2</button>
    <button onclick="append('3')">3</button>
    <button class="operator" onclick="append('-')">-</button>

    <button onclick="append('0')">0</button>

```

```

<button onclick="append('.')">.</button>
<button class="equal" onclick="calculate()">=</button>
<button class="operator" onclick="append('+')">+</button>

<button class="clear" onclick="clearDisplay()" style="grid-column: span 4;">C</button>
</div>
</div>

<script>
const display = document.getElementById("display");

function append(value) {
  display.value += value;
}

function clearDisplay() {
  display.value = "";
}

function calculate() {
  try {
    display.value = eval(display.value);
  } catch {
    display.value = "Error";
  }
}
</script>
</body>
</html>

```

8. Countdown Timer

- Create a countdown timer that takes user input (minutes/seconds) and counts down to zero, displaying an alert when done.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Countdown Timer</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f3f4f6;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
  </style>

```

```

.timer-container {
  background: white;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
  text-align: center;
}
input {
  width: 60px;
  padding: 10px;
  font-size: 16px;
  margin: 0 5px;
}
button {
  margin-top: 15px;
  padding: 10px 20px;
  background: #10b981;
  color: white;
  border: none;
  font-size: 16px;
  cursor: pointer;
  border-radius: 8px;
}
h2 {
  margin-top: 20px;
  font-size: 28px;
  color: #1e3a8a;
}
</style>
</head>
<body>

<div class="timer-container">
  <h1>Countdown Timer</h1>
  <div>
    <input type="number" id="minutes" placeholder="Min" min="0" />
    <input type="number" id="seconds" placeholder="Sec" min="0" />
  </div>
  <button onclick="startTimer()">Start</button>
  <h2 id="countdownDisplay">00:00</h2>
</div>

<script>
  let timerInterval;

  function startTimer() {
    clearInterval(timerInterval); // Reset if already running
    let minutes = parseInt(document.getElementById("minutes").value) || 0;
    let seconds = parseInt(document.getElementById("seconds").value) || 0;
    let totalSeconds = minutes * 60 + seconds;
  }
</script>

```

```

if (totalSeconds <= 0) {
  alert("Please enter a valid time.");
  return;
}

timerInterval = setInterval(() => {
  if (totalSeconds <= 0) {
    clearInterval(timerInterval);
    document.getElementById("countdownDisplay").textContent = "00:00";
    alert("🕒 Time's up!");
    return;
  }

  totalSeconds--;

  let m = Math.floor(totalSeconds / 60);
  let s = totalSeconds % 60;
  document.getElementById("countdownDisplay").textContent =
    `${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}`;
}, 1000);
}
</script>

</body>
</html>

```

9. Quiz App (5 Questions)

- Develop a quiz with 5 MCQs, a "Next" button, and a final score display. Use JavaScript to track answers.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Simple Quiz App</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f3f4f6;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .quiz-container {
      background: white;
      padding: 30px;
      border-radius: 15px;

```

```

    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    max-width: 500px;
    width: 100%;
  }
  h2 {
    margin-bottom: 20px;
    color: #1e3a8a;
  }
  .option {
    display: block;
    padding: 10px;
    margin-bottom: 10px;
    border: 2px solid #ccc;
    border-radius: 8px;
    cursor: pointer;
  }
  .option:hover {
    background-color: #e0f2fe;
  }
  button {
    background-color: #10b981;
    color: white;
    border: none;
    padding: 10px 20px;
    font-size: 16px;
    margin-top: 15px;
    cursor: pointer;
    border-radius: 8px;
  }
  #result {
    font-size: 20px;
    color: #1e3a8a;
    font-weight: bold;
    margin-top: 20px;
  }
</style>
</head>
<body>

<div class="quiz-container">
  <h2 id="question">Question text</h2>
  <div id="options"></div>
  <button onclick="nextQuestion()">Next</button>
  <div id="result"></div>
</div>

<script>
const quizData = [
  {
    question: "What is the capital of France?",
    options: ["Berlin", "Paris", "Madrid", "Rome"],
  }
]

```

```

    answer: "Paris"
  },
  {
    question: "Which planet is known as the Red Planet?",
    options: ["Earth", "Venus", "Mars", "Jupiter"],
    answer: "Mars"
  },
  {
    question: "Who wrote 'Romeo and Juliet'?",
    options: ["Mark Twain", "William Shakespeare", "Charles Dickens", "Jane Austen"],
    answer: "William Shakespeare"
  },
  {
    question: "Which element has the chemical symbol 'O'?",
    options: ["Gold", "Oxygen", "Iron", "Silver"],
    answer: "Oxygen"
  },
  {
    question: "What is 9 + 10?",
    options: ["21", "19", "20", "18"],
    answer: "19"
  }
];

```

```
let currentQuestion = 0;
```

```
let score = 0;
```

```

function loadQuestion() {
  const questionEl = document.getElementById("question");
  const optionsEl = document.getElementById("options");
  const q = quizData[currentQuestion];

  questionEl.textContent = `Q${currentQuestion + 1}. ${q.question}`;
  optionsEl.innerHTML = "";

  q.options.forEach(option => {
    const div = document.createElement("div");
    div.className = "option";
    div.textContent = option;
    div.onclick = () => {
      document.querySelectorAll(".option").forEach(opt => opt.style.background = "");
      div.style.background = "#bbf7d0";
      div.dataset.selected = "true";
    };
    optionsEl.appendChild(div);
  });
}

```

```

function nextQuestion() {
  const selected = document.querySelector('.option[style*="background"]');
  if (!selected) {

```

```

        alert("Please select an answer.");
        return;
    }

    const selectedAnswer = selected.textContent;
    if (selectedAnswer === quizData[currentQuestion].answer) {
        score++;
    }

    currentQuestion++;
    if (currentQuestion < quizData.length) {
        loadQuestion();
    } else {
        document.querySelector(".quiz-container").innerHTML = `
        <h2>Quiz Completed!</h2>
        <div id="result">Your Score: ${score} / ${quizData.length}</div>
        `;
    }
}

// Load the first question
loadQuestion();
</script>

</body>
</html>

```

10. Random Password Generator

- Generate random passwords based on user-selected criteria (length, uppercase, symbols).
Display the password on the screen.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Password Generator</title>
</style>
  body {
    font-family: Arial, sans-serif;
    background: #f3f4f6;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
  }
  .generator {
    background: white;
    padding: 30px;
    border-radius: 15px;
  }

```

```

    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    text-align: center;
    max-width: 400px;
    width: 100%;
}
h2 {
    color: #1e3a8a;
}
input[type="number"] {
    width: 80px;
    padding: 8px;
    margin: 10px;
}
label {
    display: block;
    margin: 8px 0;
}
button {
    margin-top: 15px;
    padding: 10px 20px;
    background: #10b981;
    color: white;
    border: none;
    font-size: 16px;
    cursor: pointer;
    border-radius: 8px;
}
#passwordOutput {
    margin-top: 20px;
    font-size: 18px;
    font-weight: bold;
    color: #2563eb;
    word-wrap: break-word;
}
</style>
</head>
<body>

```

```

<div class="generator">
  <h2>🔒 Password Generator</h2>
  <label>
    Length:
    <input type="number" id="length" min="4" max="30" value="12" />
  </label>
  <label><input type="checkbox" id="uppercase" checked> Include Uppercase</label>
  <label><input type="checkbox" id="numbers" checked> Include Numbers</label>
  <label><input type="checkbox" id="symbols" checked> Include Symbols</label>
  <button onclick="generatePassword()">Generate Password</button>
  <div id="passwordOutput"></div>
</div>

```

```

<script>
function generatePassword() {
  const length = document.getElementById("length").value;
  const useUpper = document.getElementById("uppercase").checked;
  const useNumbers = document.getElementById("numbers").checked;
  const useSymbols = document.getElementById("symbols").checked;

  const lower = "abcdefghijklmnopqrstuvwxyz";
  const upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  const numbers = "0123456789";
  const symbols = "!@#$%^&*()_+-=[]{}|;:,.<>?";

  let allChars = lower;
  if (useUpper) allChars += upper;
  if (useNumbers) allChars += numbers;
  if (useSymbols) allChars += symbols;

  if (length < 4 || allChars.length === 0) {
    alert("Please enter a valid length and select at least one character type.");
    return;
  }

  let password = "";
  for (let i = 0; i < length; i++) {
    const randIndex = Math.floor(Math.random() * allChars.length);
    password += allChars[randIndex];
  }

  document.getElementById("passwordOutput").textContent = password;
}
</script>

</body>
</html>

```

11. React Counter App

- Build a counter with "Increment," "Decrement," and "Reset" buttons using React state management.

CounterApp.jsx

```

import React, { useState } from 'react';

const CounterApp = () => {
  const [count, setCount] = useState(0);

  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);
  const reset = () => setCount(0);

```

```

return (
  <div style={styles.container}>
    <h1> 🚀 React Counter App</h1>
    <p style={styles.counter}>{count}</p>
    <div>
      <button style={styles.button} onClick={increment}>Increment</button>
      <button style={styles.button} onClick={decrement}>Decrement</button>
      <button style={styles.button} onClick={reset}>Reset</button>
    </div>
  </div>
);
};

const styles = {
  container: {
    textAlign: 'center',
    marginTop: '100px',
    fontFamily: 'Arial, sans-serif'
  },
  counter: {
    fontSize: '48px',
    margin: '20px 0'
  },
  button: {
    margin: '5px',
    padding: '10px 20px',
    fontSize: '16px',
    backgroundColor: '#1e3a8a',
    color: 'white',
    border: 'none',
    borderRadius: '8px',
    cursor: 'pointer'
  }
};

export default CounterApp;

```

App.js

```

import React from 'react';
import CounterApp from './CounterApp';

function App() {
  return (
    <div>
      <CounterApp />
    </div>
  );
}

export default App;

```

12. React Todo List

- Create a to-do list app in React with add/delete functionality and local storage support.

App.js

```
import React, { useState, useEffect } from 'react';

function App() {
  const [task, setTask] = useState("");
  const [todos, setTodos] = useState(() => {
    const saved = localStorage.getItem('todos');
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem('todos', JSON.stringify(todos));
  }, [todos]);

  const addTask = () => {
    if (task.trim()) {
      setTodos([...todos, task]);
      setTask("");
    }
  };

  const deleteTask = (index) => {
    const updated = todos.filter((_, i) => i !== index);
    setTodos(updated);
  };

  return (
    <div style={styles.container}>
      <h2> 📌 React To-Do List</h2>
      <div style={styles.inputRow}>
        <input
          style={styles.input}
          value={task}
          onChange={(e) => setTask(e.target.value)}
          placeholder="Enter a task"
        />
        <button onClick={addTask} style={styles.addBtn}>Add</button>
      </div>
      <ul style={styles.list}>
        {todos.map((item, index) => (
          <li key={index} style={styles.listItem}>
            {item}
            <button onClick={() => deleteTask(index)} style={styles.delBtn}> ✖ </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
    ))}
  </ul>
</div>
);
}
```

```
const styles = {
  container: {
    maxWidth: '500px',
    margin: '80px auto',
    padding: '20px',
    textAlign: 'center',
    borderRadius: '12px',
    backgroundColor: '#f9fafb',
    fontFamily: 'Arial, sans-serif',
    boxShadow: '0 0 10px rgba(0,0,0,0.1)'
  },
  inputRow: {
    display: 'flex',
    justifyContent: 'center',
    marginBottom: '20px'
  },
  input: {
    padding: '10px',
    fontSize: '16px',
    width: '70%',
    border: '1px solid #ccc',
    borderRadius: '6px 0 0 6px'
  },
  addBtn: {
    padding: '10px 20px',
    fontSize: '16px',
    border: 'none',
    backgroundColor: '#10b981',
    color: 'white',
    cursor: 'pointer',
    borderRadius: '0 6px 6px 0'
  },
  list: {
    listStyle: 'none',
    padding: 0
  },
  listItem: {
    backgroundColor: '#fff',
    margin: '10px 0',
    padding: '12px',
    borderRadius: '8px',
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center'
  },
}
```

```

delBtn: {
  background: 'none',
  border: 'none',
  fontSize: '18px',
  cursor: 'pointer'
}
};

export default App;

```

13. Weather App (Static Data)

- Display weather information (hardcoded) for a city using React components and props.

App.js

```

import React from 'react';

// WeatherCard Component to display weather information
const WeatherCard = ({ city, temperature, description, humidity, windSpeed }) => {
  return (
    <div style={styles.card}>
      <h2>{city}</h2>
      <p><strong>Temperature:</strong> {temperature}°C</p>
      <p><strong>Condition:</strong> {description}</p>
      <p><strong>Humidity:</strong> {humidity}%</p>
      <p><strong>Wind Speed:</strong> {windSpeed} km/h</p>
    </div>
  );
};

// Main App component
function App() {
  // Hardcoded weather data
  const weatherData = {
    city: 'New York',
    temperature: 22,
    description: 'Sunny',
    humidity: 55,
    windSpeed: 15
  };

  return (
    <div style={styles.container}>
      <h1>🌤️ Weather App</h1>
      <WeatherCard {...weatherData} />
    </div>
  );
}

```

```

const styles = {
  container: {
    textAlign: 'center',
    marginTop: '50px',
    fontFamily: 'Arial, sans-serif'
  },
  card: {
    backgroundColor: '#f0f4f8',
    padding: '20px',
    marginTop: '20px',
    borderRadius: '8px',
    boxShadow: '0 4px 8px rgba(0, 0, 0, 0.1)',
    maxWidth: '400px',
    margin: 'auto'
  }
};

export default App;

```

14. React Form Validation

- Implement a registration form with validation (email, password) and error messages using React hooks.

App.js

```

import React, { useState } from 'react';

// Main App Component
function App() {
  const [formData, setFormData] = useState({
    email: '',
    password: ''
  });

  const [errors, setErrors] = useState({
    email: '',
    password: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({
      ...formData,
      [name]: value
    });
  };

  const validateForm = () => {

```

```

const newErrors = {};

// Validate email
if (!formData.email) {
  newErrors.email = 'Email is required';
} else if (!/\S+@\S+\.\S+/.test(formData.email)) {
  newErrors.email = 'Email is not valid';
}

// Validate password
if (!formData.password) {
  newErrors.password = 'Password is required';
} else if (formData.password.length < 6) {
  newErrors.password = 'Password must be at least 6 characters';
}

setErrors(newErrors);
return Object.keys(newErrors).length === 0;
};

const handleSubmit = (e) => {
  e.preventDefault();
  if (validateForm()) {
    alert('Form submitted successfully');
    // Reset form data
    setFormData({
      email: '',
      password: ''
    });
    setErrors({});
  }
};

return (
  <div style={styles.container}>
    <h2>🚀 Register</h2>
    <form onSubmit={handleSubmit} style={styles.form}>
      <div style={styles.formGroup}>
        <label htmlFor="email">Email:</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
          style={styles.input}
        />
        {errors.email && <span style={styles.error}>{errors.email}</span>}
      </div>

      <div style={styles.formGroup}>
        <label htmlFor="password">Password:</label>

```

```

    <input
      type="password"
      name="password"
      value={formData.password}
      onChange={handleChange}
      style={styles.input}
    />
    {errors.password && <span style={styles.error}>{errors.password}</span>}
  </div>

  <button type="submit" style={styles.submitButton}>Register</button>
</form>
</div>
);
}

```

```

const styles = {
  container: {
    maxWidth: '500px',
    margin: '50px auto',
    padding: '20px',
    textAlign: 'center',
    borderRadius: '8px',
    backgroundColor: '#f3f4f6',
    boxShadow: '0 4px 8px rgba(0, 0, 0, 0.1)',
    fontFamily: 'Arial, sans-serif'
  },
  form: {
    display: 'flex',
    flexDirection: 'column',
    alignItems: 'center'
  },
  formGroup: {
    marginBottom: '15px',
    width: '100%'
  },
  input: {
    width: '80%',
    padding: '10px',
    fontSize: '16px',
    margin: '5px 0',
    borderRadius: '5px',
    border: '1px solid #ccc'
  },
  error: {
    color: 'red',
    fontSize: '12px',
    marginTop: '5px'
  },
  submitButton: {
    padding: '10px 20px',
  }
}

```

```

    fontSize: '16px',
    border: 'none',
    backgroundColor: '#10b981',
    color: 'white',
    cursor: 'pointer',
    borderRadius: '6px',
    width: '80%',
    marginTop: '20px'
  }
};

export default App;

```

15. React Photo Gallery

- Build a gallery that displays images in a grid. Add a button to filter images by category.

App.js

```

import React, { useState } from 'react';
import './App.css';

// Sample image data
const imageData = [
  { id: 1, src: 'https://via.placeholder.com/150', category: 'Nature' },
  { id: 2, src: 'https://via.placeholder.com/150', category: 'City' },
  { id: 3, src: 'https://via.placeholder.com/150', category: 'Animals' },
  { id: 4, src: 'https://via.placeholder.com/150', category: 'Nature' },
  { id: 5, src: 'https://via.placeholder.com/150', category: 'City' },
  { id: 6, src: 'https://via.placeholder.com/150', category: 'Animals' }
];

function App() {
  const [selectedCategory, setSelectedCategory] = useState('All');

  const categories = ['All', ...new Set(imageData.map(img => img.category))];

  const filteredImages =
    selectedCategory === 'All'
      ? imageData
      : imageData.filter(img => img.category === selectedCategory);

  return (
    <div className="App">
      <h1> 📷 React Photo Gallery</h1>

      <div className="buttons">
        {categories.map((cat, idx) => (
          <button key={idx} onClick={() => setSelectedCategory(cat)}>
            {cat}
          </button>
        ))}
      </div>

      <div>
        {filteredImages.map(img => (
          <img alt={img.src} key={img.id} />
        ))}
      </div>
    </div>
  );
}

```

```

        </button>
      )}
    </div>

    <div className="gallery">
      {filteredImages.map(img => (
        <div key={img.id} className="photo-card">
          <img src={img.src} alt={`Image ${img.id}`} />
          <p>{img.category}</p>
        </div>
      ))}
    </div>
  </div>
);
}

```

```
export default App;
```

App.css

```

.App {
  text-align: center;
  padding: 2rem;
  font-family: sans-serif;
  background-color: #f5f5f5;
}

h1 {
  color: #1e3a8a;
}

.buttons {
  margin: 20px 0;
}

button {
  margin: 0 10px;
  padding: 10px 15px;
  background-color: #10b981;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background-color: #059669;
}

.gallery {
  display: grid;

```

```

    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 20px;
    margin-top: 20px;
}

.photo-card {
    background: white;
    padding: 10px;
    border-radius: 8px;
    box-shadow: 0px 2px 6px rgba(0, 0, 0, 0.1);
}

.photo-card img {
    width: 100%;
    border-radius: 4px;
}

.photo-card p {
    margin: 8px 0 0;
    font-size: 14px;
    color: #555;
}

```

16. Angular Hello World with Data Binding

- Create a simple Angular app with two-way data binding (e.g., input field updates a displayed message).

app.component.ts

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  message = 'Angular!';
}

```

app.component.html

Open src/app/app.component.html:

```

<div style="text-align:center; margin-top: 50px;">
  <h1>Hello {{ message }}</h1>

  <input [(ngModel)]="message" placeholder="Enter your message" />
</div>

```

Open src/app/app.module.ts and add FormsModule:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms'; // ♦ Import this

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule], // ♦ Add FormsModule here
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

17. Angular Student Record Manager

- Build a form to add/edit student records (name, roll no, grade) and display them in a table.

1. Create the Angular App

```
ng new student-record-manager
cd student-record-manager
ng serve
```

app.module.ts

Edit src/app/app.module.ts:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

app.component.ts

```
import { Component } from '@angular/core';

interface Student {
```

```

name: string;
rollNo: number;
grade: string;
}

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  student: Student = { name: "", rollNo: 0, grade: "" };
  students: Student[] = [];
  editingIndex: number | null = null;

  addOrUpdateStudent() {
    if (this.editingIndex !== null) {
      this.students[this.editingIndex] = { ...this.student };
      this.editingIndex = null;
    } else {
      this.students.push({ ...this.student });
    }
    this.resetForm();
  }

  editStudent(index: number) {
    this.student = { ...this.students[index] };
    this.editingIndex = index;
  }

  deleteStudent(index: number) {
    this.students.splice(index, 1);
  }

  resetForm() {
    this.student = { name: "", rollNo: 0, grade: "" };
  }
}

```

app.component.html

```

<div class="container mt-4">
  <h2> 🎓 Student Record Manager</h2>

  <form (ngSubmit)="addOrUpdateStudent()">
    <div class="form-group mb-2">
      <label>Name:</label>
      <input class="form-control" [(ngModel)]="student.name" name="name" required />
    </div>
    <div class="form-group mb-2">
      <label>Roll No:</label>

```

```

        <input class="form-control" type="number" [(ngModel)]="student.rollNo" name="rollNo"
required />
    </div>
    <div class="form-group mb-2">
        <label>Grade:</label>
        <input class="form-control" [(ngModel)]="student.grade" name="grade" required />
    </div>
    <button class="btn btn-primary mt-2" type="submit">
        {{ editingIndex !== null ? 'Update' : 'Add' }} Student
    </button>
</form>

<hr />

<table class="table table-bordered mt-4" *ngIf="students.length > 0">
    <thead>
        <tr>
            <th>Name</th>
            <th>Roll No</th>
            <th>Grade</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <tr *ngFor="let stu of students; let i = index">
            <td>{{ stu.name }}</td>
            <td>{{ stu.rollNo }}</td>
            <td>{{ stu.grade }}</td>
            <td>
                <button class="btn btn-warning btn-sm" (click)="editStudent(i)">Edit</button>
                <button class="btn btn-danger btn-sm ms-1" (click)="deleteStudent(i)">Delete</button>
            </td>
        </tr>
    </tbody>
</table>
</div>

```

app.component.css

```

.container {
    max-width: 600px;
    margin: auto;
    padding: 20px;
}

```

18. Angular Shopping Cart

- Implement a cart system where users can add/remove items and see the total price.

app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

app.component.ts

```
import { Component } from '@angular/core';

interface Product {
  id: number;
  name: string;
  price: number;
}

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  products: Product[] = [
    { id: 1, name: 'Laptop', price: 800 },
    { id: 2, name: 'Headphones', price: 150 },
    { id: 3, name: 'Mouse', price: 40 }
  ];

  cart: Product[] = [];

  addToCart(product: Product) {
    this.cart.push(product);
  }

  removeFromCart(index: number) {
    this.cart.splice(index, 1);
  }
}
```

```

getTotal(): number {
  return this.cart.reduce((sum, item) => sum + item.price, 0);
}
}

```

app.component.html

```

<div class="container mt-4">
  <h2>🛒 Shopping Cart</h2>

  <div class="row">
    <div class="col-md-6">
      <h4>Products</h4>
      <ul class="list-group">
        <li class="list-group-item" *ngFor="let product of products">
          {{ product.name }} - ${{ product.price }}
          <button class="btn btn-sm btn-primary float-end" (click)="addToCart(product)">Add</button>
        </li>
      </ul>
    </div>

    <div class="col-md-6">
      <h4>Your Cart</h4>
      <ul class="list-group">
        <li class="list-group-item" *ngFor="let item of cart; let i = index">
          {{ item.name }} - ${{ item.price }}
          <button class="btn btn-sm btn-danger float-end"
            (click)="removeFromCart(i)">Remove</button>
        </li>
      </ul>
      <div class="mt-3">
        <strong>Total:</strong> ${{ getTotal() }}
      </div>
    </div>
  </div>
</div>

```

19. Angular Quiz App

- Develop a 5-question quiz with navigation between questions and a results page.

app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';
```

```
import { AppComponent } from './app.component';
```

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

app.component.ts

```
import { Component } from '@angular/core';
```

```
interface Question {
  question: string;
  options: string[];
  answer: string;
}
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  currentQuestionIndex = 0;
  selectedAnswers: string[] = [];
  showResults = false;
  score = 0;

  questions: Question[] = [
    {
      question: 'What is the capital of France?',
      options: ['Paris', 'London', 'Berlin', 'Madrid'],
      answer: 'Paris'
    },
    {
      question: '2 + 2 = ?',
      options: ['3', '4', '5', '2'],
      answer: '4'
    },
    {
      question: 'What is the color of the sky?',
      options: ['Green', 'Blue', 'Red', 'Yellow'],
      answer: 'Blue'
    },
  ]
}
```

```

    question: 'Angular is developed by?',
    options: ['Facebook', 'Google', 'Microsoft', 'Amazon'],
    answer: 'Google'
  },
  {
    question: 'Which is a frontend framework?',
    options: ['Django', 'Laravel', 'React', 'Flask'],
    answer: 'React'
  }
];

selectOption(option: string) {
  this.selectedAnswers[this.currentQuestionIndex] = option;
}

nextQuestion() {
  if (this.currentQuestionIndex < this.questions.length - 1) {
    this.currentQuestionIndex++;
  } else {
    this.calculateScore();
    this.showResults = true;
  }
}

prevQuestion() {
  if (this.currentQuestionIndex > 0) {
    this.currentQuestionIndex--;
  }
}

calculateScore() {
  this.score = this.questions.reduce((acc, question, i) => {
    return acc + (this.selectedAnswers[i] === question.answer ? 1 : 0);
  }, 0);
}
}

```

app.component.html

```

<div class="container mt-5">
  <h2 class="text-center">🧠 Angular Quiz App</h2>

  <div *ngIf="!showResults">
    <h4>Question {{ currentQuestionIndex + 1 }} of {{ questions.length }}</h4>
    <p><strong>{{ questions[currentQuestionIndex].question }}</strong></p>

    <div *ngFor="let option of questions[currentQuestionIndex].options">
      <label>
        <input type="radio"
          name="option"
          [value]="option"
          [(ngModel)]="selectedAnswers[currentQuestionIndex]">

```

```

    {{ option }}
  </label>
</div>

<div class="mt-3">
  <button class="btn btn-secondary me-2" (click)="prevQuestion()"
[disabled]="currentQuestionIndex === 0">Previous</button>
  <button class="btn btn-primary" (click)="nextQuestion()">
    {{ currentQuestionIndex === questions.length - 1 ? 'Finish' : 'Next' }}
  </button>
</div>
</div>

<div *ngIf="showResults">
  <h3>Your Score: {{ score }}/{{ questions.length }}</h3>
  <ul>
    <li *ngFor="let q of questions; let i = index">
      {{ q.question }} <br />
      <strong>Your answer:</strong> {{ selectedAnswers[i] }} <br />
      <strong>Correct answer:</strong> {{ q.answer }}
      <hr />
    </li>
  </ul>
  <button class="btn btn-success" (click)="currentQuestionIndex = 0; showResults = false;
selectedAnswers = []; score = 0;">Retake Quiz</button>
</div>
</div>

```

20. Angular Movie Search (Static Data)

- Display a list of movies (hardcoded) with search functionality to filter by title.

app.module.ts

```

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

app.component.ts

```

import { Component } from '@angular/core';

```

```

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  searchText = "";

  movies = [
    { title: 'The Shawshank Redemption', year: 1994 },
    { title: 'The Godfather', year: 1972 },
    { title: 'Inception', year: 2010 },
    { title: 'Interstellar', year: 2014 },
    { title: 'The Dark Knight', year: 2008 },
    { title: 'Forrest Gump', year: 1994 },
    { title: 'Pulp Fiction', year: 1994 },
  ];

  get filteredMovies() {
    return this.movies.filter(movie =>
      movie.title.toLowerCase().includes(this.searchText.toLowerCase())
    );
  }
}

```

app.component.html

```

<div class="container mt-5">
  <h2 class="text-center"> 🎬 Angular Movie Search</h2>

  <input
    type="text"
    class="form-control my-3"
    placeholder="Search by movie title..."
    [(ngModel)]="searchText"
  />

  <ul class="list-group">
    <li
      *ngFor="let movie of filteredMovies"
      class="list-group-item d-flex justify-content-between align-items-center"
    >
      {{ movie.title }}
      <span class="badge bg-primary">{{ movie.year }}</span>
    </li>
    <li *ngIf="filteredMovies.length === 0" class="list-group-item text-muted">
      No movies found.
    </li>
  </ul>
</div>

```

21. REST API for Bookstore

- Create a Node.js/Express API with GET/POST endpoints for managing a list of books (title, author). Use Postman to test.

```
// index.js

// 1. Import the Express module
const express = require('express');

// 2. Create an Express application
const app = express();

// 3. Define a port number (you can change it)
const PORT = 3000;

// 4. Use express.json() middleware to parse JSON request bodies
app.use(express.json());

// 5. In-memory array to store books (acting as a temporary database)
let books = [];

// 6. GET route to retrieve all books
app.get('/api/books', (req, res) => {
  res.json(books); // Respond with the array of books in JSON format
});

// 7. POST route to add a new book
app.post('/api/books', (req, res) => {
  const { title, author } = req.body; // Extract title and author from the request body

  if (!title || !author) {
    // If title or author is missing, send a 400 (Bad Request) response
    return res.status(400).json({ message: 'Title and author are required' });
  }

  // Create a new book object
  const newBook = { id: books.length + 1, title, author };

  // Add the new book to the array
  books.push(newBook);

  // Respond with a success message and the added book
  res.status(201).json({ message: 'Book added successfully', book: newBook });
});

// 8. Start the server and listen on the defined port
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

22. User Login System (Mock)

- Build a simple login system with hardcoded credentials. Return "success" or "failure" based on input.

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Mock Login</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="login-container">
    <h2>Login</h2>
    <input type="text" id="username" placeholder="Username" />
    <input type="password" id="password" placeholder="Password" />
    <button onclick="login()">Login</button>
    <p id="result"></p>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  font-family: sans-serif;
  background: #f0f0f0;
}

.login-container {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

input {
  display: block;
  margin: 1rem 0;
  padding: 0.5rem;
```

```

width: 100%;
}

button {
padding: 0.5rem 1rem;
background: #1e90ff;
color: white;
border: none;
cursor: pointer;
}

#result {
margin-top: 1rem;
font-weight: bold;
}

```

script.js

```

function login() {
  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;
  const result = document.getElementById("result");

  // Hardcoded credentials
  const validUser = "admin";
  const validPass = "12345";

  if (username === validUser && password === validPass) {
    result.textContent = "☑ Login Successful";
    result.style.color = "green";
  } else {
    result.textContent = "✗ Login Failed";
    result.style.color = "red";
  }
}

```

23. CRUD API for Notes

- Implement a REST API to create, read, update, and delete notes (title, content) using Express and MongoDB.

```

mkdir notes-api
cd notes-api
npm init -y
npm install express mongoose body-parser cors

```

FOLDER STRUCTURE

```

notes-api/
|

```

```
├── models/
│   └── Note.js
├── routes/
│   └── notes.js
└── server.js
```

models/Note.js

```
const mongoose = require('mongoose');

const noteSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true
  },
  content: {
    type: String,
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model('Note', noteSchema);
```

routes/notes.js

```
const express = require('express');
const router = express.Router();
const Note = require('../models/Note');

// CREATE
router.post('/', async (req, res) => {
  try {
    const { title, content } = req.body;
    const newNote = new Note({ title, content });
    const saved = await newNote.save();
    res.status(201).json(saved);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// READ ALL
router.get('/', async (req, res) => {
  try {
    const notes = await Note.find();
    res.json(notes);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

```

// READ SINGLE
router.get('/:id', async (req, res) => {
  try {
    const note = await Note.findById(req.params.id);
    if (!note) return res.status(404).json({ message: 'Note not found' });
    res.json(note);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// UPDATE
router.put('/:id', async (req, res) => {
  try {
    const { title, content } = req.body;
    const updated = await Note.findByIdAndUpdate(
      req.params.id,
      { title, content },
      { new: true }
    );
    res.json(updated);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const deleted = await Note.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ message: 'Note not found' });
    res.json({ message: 'Note deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

```

module.exports = router;

```

server.js

```

const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const cors = require('cors');

const app = express();
const PORT = 5000;

// Middleware
app.use(cors());

```

```

app.use(bodyParser.json());

// Routes
const notesRoutes = require('./routes/notes');
app.use('/api/notes', notesRoutes);

// MongoDB Connection
mongoose.connect('mongodb://localhost:27017/notesdb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('Connected to MongoDB');
  app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
}).catch(err => console.error('MongoDB connection error:', err));

```

Run the command: `node server.js`

24. URL Shortener (Basic)

- Create an API that takes a long URL and returns a short code. Store mappings in MongoDB.

url-shortener/

|

|— models/

| └─ Url.js

|— routes/

| └─ shortener.js

|— server.js

models/Url.js

```

const mongoose = require('mongoose');

const urlSchema = new mongoose.Schema({
  originalUrl: { type: String, required: true },
  shortCode: { type: String, required: true, unique: true },
  createdAt: { type: Date, default: Date.now }
});

```

```
});
```

```
module.exports = mongoose.model('Url', urlSchema);
```

routes/shortener.js

```
const express = require('express');
```

```
const router = express.Router();
```

```
const { nanoid } = require('nanoid');
```

```
const Url = require('../models/Url');
```

```
// Base URL (optional customization)
```

```
const BASE_URL = 'http://localhost:5000/';
```

```
// Create short URL
```

```
router.post('/shorten', async (req, res) => {
```

```
  const { originalUrl } = req.body;
```

```
  if (!originalUrl) return res.status(400).json({ error: 'URL is required' });
```

```
  try {
```

```
    // Generate short code
```

```
    const shortCode = nanoid(6);
```

```
    // Save to DB
```

```
    const newUrl = new Url({ originalUrl, shortCode });
```

```
    await newUrl.save();
```

```
    res.json({ shortUrl: BASE_URL + shortCode });
```

```
  } catch (err) {
```

```
    res.status(500).json({ message: err.message });
```

```
  }
```

```
});
```

```
// Redirect using short code
```

```
router.get('/:shortCode', async (req, res) => {
```

```
  try {
```

```
    const url = await Url.findOne({ shortCode: req.params.shortCode });
```

```
    if (url) {
```

```
      return res.redirect(url.originalUrl);
```

```
    } else {
```

```
      return res.status(404).json({ error: 'Short URL not found' });
```

```
    }
```

```
  } catch (err) {
```

```
    res.status(500).json({ message: err.message });
```

```
  }
```

```
});
```

```
module.exports = router;
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const cors = require('cors');

const app = express();
const PORT = 5000;

// Middleware
app.use(cors());
app.use(bodyParser.json());

// Routes
const shortenerRoutes = require('./routes/shortener');
app.use('/', shortenerRoutes);

// MongoDB
mongoose.connect('mongodb://localhost:27017/urlshortener', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB Connected');
  app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
}).catch(err => console.error(err));
```

How to Test

1. **POST** to `http://localhost:5000/shorten` with JSON:
`{ "originalUrl": "https://example.com/some/long/url" }`

25. Contact Form Backend

- Develop an Express API to save contact form submissions (name, email, message) to MongoDB.

contact-form-backend/

```
|
|— models/
|   └─ Contact.js
|— routes/
|   └─ contact.js
|— server.js
```

`mkdir contact-form-backend`

```
cd contact-form-backend
npm init -y
npm install express mongoose cors body-parser
```

models/Contact.js

```
const mongoose = require('mongoose');

const contactSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  message: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Contact', contactSchema);
```

routes/contact.js

```
const express = require('express');
const router = express.Router();
const Contact = require('../models/Contact');

// POST /contact - Save form submission
router.post('/', async (req, res) => {
  const { name, email, message } = req.body;

  if (!name || !email || !message) {
    return res.status(400).json({ error: 'All fields are required.' });
  }

  try {
    const newContact = new Contact({ name, email, message });
    await newContact.save();
    res.status(201).json({ message: 'Contact form submitted successfully.' });
  } catch (err) {
    res.status(500).json({ error: 'Server error. Please try again.' });
  }
});

module.exports = router;
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const bodyParser = require('body-parser');
```

```
const app = express();
const PORT = 5000;

// Middleware
app.use(cors());
app.use(bodyParser.json());

// Routes
const contactRoute = require('./routes/contact');
app.use('/contact', contactRoute);

// MongoDB Connection
mongoose.connect('mongodb://localhost:27017/contactform', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB connected');
  app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
}).catch(err => console.error(err));
```

☒ How to Test

Send a **POST request** to:

bash

<http://localhost:5000/contact>

With JSON Body:

```
json
{
  "name": "John Doe",
  "email": "john@example.com",
  "message": "Hello, this is a test message."
}
```

☒ You'll get:

```
json
{ "message": "Contact form submitted successfully." }
```